Cryptography Messenger Client Questions

Collaborators:

- Brenton Lian
- Akksharvan Senthilkumar

Question 1: In our implementation, Alice and Bob increment their Diffie-Hellman ratchets every time they exchange messages. Could the protocol be modified to have them increment the DH ratchets once every ten messages without compromising confidentiality against an eavesdropper (i.e., semantic security)? Why or why not?

Yes, updating the DH ratchet once every ten messages would still preserve semantic security against a passive eavesdropper who has not compromised the secret state. The symmetric-key ratchet would still advance for every message, deriving a fresh pseudorandom message key using HMAC-SHA256, with AES-GCM used correctly under each key. However, less frequent DH ratcheting would delay the introduction of fresh shared secrets and could expose more future messages after a state compromise, delaying break-in recovery.

Question 2: What if they never update their DH keys at all? Please explain the security consequences of this change with regard to forward secrecy and break-in recovery.

If only a current chain key is compromised, the one-way symmetric ratchet still prevents the attacker from deriving earlier, deleted chain and message keys. However, in this assignment's initialization, compromising a retained initial DH private key lets the attacker recompute the initial shared secret from the peer's public key and reconstruct past message keys, defeating forward secrecy against that compromise. Break-in recovery is also lost: an attacker who learns the current chain keys can derive future message keys indefinitely because no fresh DH secret is introduced.

Question 3: The message reporting feature included in our messaging scheme is not adequate for use on an actual messaging platform. What is one shortcoming of our approach?

The reporting system cannot verify whether the alleged sender actually authored the reported message. Because the reporting client supplies both the sender's name and the message contents, a malicious user can fabricate a message and attribute it to someone else. Authenticated encryption protects the report's confidentiality and detects modification of the encrypted report, but it does not prove that the accused user sent the reported text.

Question 4: Our messaging system relies on the platform to verify the authenticity of users' public keys, but we are also trying to give users confidentiality from the platform. Using this partial trust we place in it, how could a malicious platform learn the contents of a message one user is sending to another?

A malicious platform could replace Bob's public key in the certificate given to Alice with a platform-controlled key and sign the fraudulent certificate using its legitimate signing key. It could similarly substitute a platform-controlled key for Alice's key in the certificate given to Bob. The platform could then establish two separate Double Ratchet sessions, impersonating Bob to Alice and Alice to Bob, and decrypt and re-encrypt messages as it relays them. Both clients would accept the fraudulent certificates because they trust the platform's signing key.

Question 5: How do end-to-end encrypted messaging apps help users avoid this problem? Feel free to look this up or explore one of these messaging apps to answer this question

End-to-end encrypted messaging apps such as Signal let users independently verify their contacts' encryption keys by comparing safety numbers or scanning QR codes through a trusted channel, such as in person. Signal also indicates when a safety number changes; this warrants verification but can result from a legitimate change, such as reinstalling the app, rather than an attack. WhatsApp uses key transparency to make its public-key directory auditable and help detect inconsistent or unauthorized key substitutions. These mechanisms reduce reliance on the platform as the sole authority for public-key authenticity